

**Similarly some examples:**

```
>>> x=[m for m in range(8)]
```

```
>>> print(x)
```

```
[0, 1, 2, 3, 4, 5, 6, 7]
```

```
>>> x=[z**2 for z in range(10) if z>4]
```

```
>>> print(x)
```

```
[25, 36, 49, 64, 81]
```

```
>>> x=[x ** 2 for x in range (1, 11) if x % 2 == 1]
```

```
>>> print(x)
```

```
[1, 9, 25, 49, 81]
```

```
>>> a=5
```

```
>>> table = [[a, b, a * b] for b in range(1, 11)]
```

```
>>> for i in table:
```

```
    print(i)
```

```
[5, 1, 5]
```

```
[5, 2, 10]
```

```
[5, 3, 15]
```

```
[5, 4, 20]
```

```
[5, 5, 25]
```

```
[5, 6, 30]
```

```
[5, 7, 35]
```

```
[5, 8, 40]
```

```
[5, 9, 45]
```

```
[5, 10, 50]
```

**Tuples:**

A tuple is a collection which is ordered and unchangeable. In Python tuples are written with round brackets.

- Supports all operations for sequences.
- Immutable, but member objects may be mutable.
- If the contents of a list shouldn't change, use a tuple to prevent items from

accidentally being added, changed, or deleted.

- Tuples are more efficient than list due to python's implementation.

We can construct tuple in many ways:

```
X=() #no item tuple
```

```
X=(1,2,3)
```

```
X=tuple(list1)
```

```
X=1,2,3,4
```

### Example:

```
>>> x=(1,2,3)
```

```
>>> print(x)
```

```
(1, 2, 3)
```

```
>>> x
```

```
(1, 2, 3)
```

```
-----
```

```
>>> x=()
```

```
>>> x
```

```
()
```

```
-----
```

```
>>> x=[4,5,66,9]
```

```
>>> y=tuple(x)
```

```
>>> y
```

```
(4, 5, 66, 9)
```

```
-----
```

```
>>> x=1,2,3,4
```

```
>>> x
```

```
(1, 2, 3, 4)
```

Some of the operations of tuple are:

- Access tuple items
- Change tuple items
- Loop through a tuple
- Count()
- Index()
- Length()

**Access tuple items:** Access tuple items by referring to the index number, inside square brackets

```
>>> x=('a','b','c','g')
```

```
>>> print(x[2])
```

c

**Change tuple items:** Once a tuple is created, you cannot change its values. Tuples are unchangeable.

```
>>> x=(2,5,7,'4',8)
```

```
>>> x[1]=10
```

Traceback (most recent call last):

File "<pyshell#41>", line 1, in <module>

x[1]=10

**TypeError: 'tuple' object does not support item assignment**

```
>>> x
```

```
(2, 5, 7, '4', 8) # the value is still the same
```

**Loop through a tuple:** We can loop the values of tuple using for loop

```
>>> x=4,5,6,7,2,'aa'
```

```
>>> for i in x:
```

```
    print(i)
```

4

5

6

7

2

aa

**Count ():** Returns the number of times a specified value occurs in a tuple

```
>>> x=(1,2,3,4,5,6,2,10,2,11,12,2)
```

```
>>> x.count(2)
```

4

**Index ():** Searches the tuple for a specified value and returns the position of where it was found

```
>>> x=(1,2,3,4,5,6,2,10,2,11,12,2)
>>> x.index(2)
1
```

(Or)

```
>>> x=(1,2,3,4,5,6,2,10,2,11,12,2)
>>> y=x.index(2)
>>> print(y)
1
```

**Length ():** To know the number of items or values present in a tuple, we use len().

```
>>> x=(1,2,3,4,5,6,2,10,2,11,12,2)
>>> y=len(x)
>>> print(y)
12
```

### Tuple Assignment

Python has tuple assignment feature which enables you to assign more than one variable at a time. In here, we have assigned tuple 1 with the college information like college name, year, etc. and another tuple 2 with the values in it like number (1, 2, 3... 7).

For Example,

Here is the code,

- >>> tup1 = ('mrcet', 'eng college','2004','cse', 'it','csit');
- >>> tup2 = (1,2,3,4,5,6,7);
- >>> print(tup1[0])
- mrcet
- >>> print(tup2[1:4])
- (2, 3, 4)

Tuple 1 includes list of information of mrcet

Tuple 2 includes list of numbers in it

We call the value for [0] in tuple and for tuple 2 we call the value between 1 and 4

Run the above code- It gives name mrcet for first tuple while for second tuple it gives number (2, 3, 4)

### **Tuple as return values:**

A Tuple is a comma separated sequence of items. It is created with or without (). Tuples are immutable.

# A Python program to return multiple values from a method using tuple

# This function returns a tuple

```
def fun():
```

```
    str = "mrcet college"
```

```
    x = 20
```

```
    return str, x; # Return tuple, we could also
```

```
        # write (str, x)
```

```
# Driver code to test above method
```

```
str, x = fun() # Assign returned tuple
```

```
print(str)
```

```
print(x)
```

### **Output:**

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/tupretval.py
```

```
mrcet college
```

```
20
```

- Functions can return tuples as return values.

```
def circleInfo(r):
```

```
    """ Return (circumference, area) of a circle of radius r """
```

```
    c = 2 * 3.14159 * r
```

```
    a = 3.14159 * r * r
```

```
    return (c, a)
```

```
print(circleInfo(10))
```

### **Output:**

```
C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/functupretval.py
```

```
(62.8318, 314.159)
```

```
def f(x):  
    y0 = x + 1  
    y1 = x * 3  
    y2 = y0 ** y3  
    return (y0, y1, y2)
```

### Tuple comprehension:

Tuple Comprehensions are special: The result of a tuple comprehension is special. You might expect it to produce a tuple, but what it does is produce a special "generator" object that we can iterate over.

#### For example:

```
>>> x = (i for i in 'abc') #tuple comprehension  
>>> x  
<generator object <genexpr> at 0x033EEC30>  
  
>>> print(x)  
<generator object <genexpr> at 0x033EEC30>
```

You might expect this to print as ('a', 'b', 'c') but it prints as <generator object <genexpr> at 0x02AAD710> The result of a tuple comprehension is not a tuple: it is actually a generator. The only thing that you need to know now about a generator now is that you can iterate over it, but ONLY ONCE.

So, given the code

```
>>> x = (i for i in 'abc')  
>>> for i in x:  
    print(i)
```

a  
b  
c

#### Create a list of 2-tuples like (number, square):

```
>>> z=[(x, x**2) for x in range(6)]  
>>> z  
[(0, 0), (1, 1), (2, 4), (3, 9), (4, 16), (5, 25)]
```